

patX: Interpretable Pattern Extraction for Time Series and Spatial Data

Jonas Wolber¹, Second Author², Third Author¹

¹RWTH Aachen University

²Institution Two

September 24, 2025

Abstract

We introduce **patX**, an open-source Python package that automatically discovers compact, highly predictive, and interpretable patterns from biomedical time-series and spatial data. By framing pattern discovery as a constrained optimization problem over a pool of candidate subsequences, patX learns human-readable patterns that serve as transparent input features for conventional machine-learning models. We demonstrate patX on four diverse biomedical tasks: (i) glucose forecasting (AZT1D), (ii) arrhythmia classification (MITBIH), (iii) seizure classification (Bonn EEG), and (iv) gene expression prediction from epigenetic signatures (REMC). Across four of the five tasks, patX outperforms tsfresh and convolutional neural networks, while providing direct interpretability and significant speed-ups at inference time. We also demonstrate the interpretability of patX patterns by visualizing pattern and the area at which they are active. Our tool is available at <https://github.com/Prgrmmrjns/patX> and offers a simple interface for extracting patterns from time series and spatial data.

1 Introduction

Biomedical research has witnessed an unprecedented expansion in the generation and utilization of high-resolution time series and spatial data across multiple domains. From continuous physiological monitoring through electrocardiography (ECG) and electroencephalography (EEG) to advanced spatially-resolved omics technologies, these data modalities have become fundamental to modern healthcare and biomedical discovery [20, 27].

Time series data in healthcare captures the temporal evolution of patient health, offering crucial insights into disease progression, treatment effectiveness, and physiological dynamics that static measurements cannot provide [23]. Continuous monitoring systems such as smart watches or electroencephalography devices generate vast streams of physiological signals—from heart rate variability and blood pressure fluctuations to brain activity patterns—that contain rich information about both normal physiology and pathological states [5].

The clinical applications of these data types are extensive and growing. Electrocardiography (ECG) analysis supports arrhythmia detection and cardiovascular risk stratification, with modern systems processing thousands of heartbeats to identify subtle patterns indicative of cardiac pathology [1]. EEG monitoring enables real-time seizure detection, sleep stage classification, and assessment of neurological function across various conditions [5]. In genomics and epigenomics, spatial profiling of chromatin modifications and gene expression patterns provides insights into regulatory mechanisms underlying development and disease [30].

The analysis of biomedical time series and spatial data has evolved through three different paradigms that represent fundamentally different philosophies toward feature representation and model development. Traditional approaches to biomedical signal analysis rely heavily on domain expertise to manually craft features that capture relevant temporal and spatial characteristics. This paradigm involves extracting carefully designed statistical measures, temporal properties, spectral features, and morphological characteristics based on established understanding of physiological processes [9]. For ECG analysis, clinicians and engineers have developed features capturing heart rate variability, QRS complex morphology, and ST-segment deviations [11, 3]. Similarly, EEG analysis traditionally focuses on frequency band power, coherence measures, and event-related potentials derived from decades of neurophysiology research [22].

This approach benefits significantly from the incorporation of clinical domain knowledge and provides features with clear physiological interpretations that align with medical understanding. Manual feature engineering allows for precise targeting of known pathological markers and enables clinicians to understand exactly which signal characteristics drive model predictions [11].

However, manual feature engineering suffers from several critical limitations that constrain its scalability and generalizability. The feature engineering process is labor-intensive, requiring expertise in both signal processing techniques and the specific clinical domain as well as requiring a preceding detailed exploratory data analysis [26]. This expertise requirement creates bottlenecks in model development and limits the ability to rapidly adapt to new clinical problems or signal types. Moreover, manually crafted features may miss subtle patterns that could be crucial for accurate classification or prediction, particularly in complex, high-dimensional biomedical datasets where relevant patterns may not be immediately apparent to human experts [18, 3].

Comprehensive automated feature extraction libraries address the scalability limitations of manual approaches by systematically computing large sets of time series characteristics. `tsfresh` extracts up to 1,558 features spanning statistical, spectral, and nonlinear dynamics properties, implementing extensive collections of feature calculators followed by statistical hypothesis testing for feature selection [6]. `TSFEL` offers a more streamlined approach with over 65 features across statistical, temporal, spectral, and fractal domains, emphasizing computational efficiency and user-friendly deployment [4]. `catch22` represents a fundamentally different philosophy, providing exactly 22 carefully selected features derived from a massive reduction of over 7,000 features in the `hctsa` toolbox, optimized for both performance and computational efficiency [17]. `hctsa` itself computes over 7,000 features from an interdisciplinary survey of time series analysis methods, serving as a comprehensive research tool though with significant computational requirements [10].

These libraries effectively address the scalability challenges of manual feature engineering and can uncover patterns that might be missed by domain experts. However, they introduce new challenges related to feature interpretability, computational complexity, and feature selection from high-dimensional feature spaces [13]. While individual features may have clear mathematical definitions, understanding which specific aspects of complex biomedical signals drive clinical predictions becomes difficult when models rely on hundreds or thousands of automatically extracted features [25].

Deep learning methods represent yet another paradigmatic shift toward end-to-end learning, where neural networks learn hierarchical feature representations directly from raw data without requiring explicit feature extraction steps. This approach fundamentally differs from both manual feature engineering and automated feature extraction libraries by learning data representations as an integral part of the prediction task [18].

Convolutional Neural Networks (CNNs) have demonstrated particular success in biomedical time series analysis, automatically learning relevant temporal patterns through convolutional filters that detect local structures across different time scales [29, 21]. Recurrent Neural Networks

(RNNs) and their variants, including Long Short-Term Memory (LSTMs) and Gated Recurrent Units (GRUs), excel at modeling sequential dependencies and temporal memory in physiological signals, making them well-suited for applications requiring understanding of disease progression or long-term monitoring patterns [15, 16].

However, deep learning approaches face significant challenges in biomedical applications, particularly regarding interpretability and the “black box” nature of learned representations [8]. While these methods can achieve excellent predictive performance, understanding how they arrive at their decisions remains difficult, creating barriers to clinical adoption where explainability is crucial for patient safety and regulatory approval [28, 8]. The evolution from manual feature engineering through automated extraction libraries to end-to-end deep learning reflects a fundamental trade-off between interpretability and automated pattern discovery. While manual approaches offer maximum clinical interpretability, they suffer from scalability limitations and potential blind spots in pattern recognition. Automated feature extraction libraries provide computational scalability but may generate features that lack clinical meaning. Deep learning methods offer powerful automatic feature discovery and superior performance but at the cost of interpretability.

We address this trade-off with **patX**, a method that unifies the strengths of all three approaches: automatic feature extraction without manual engineering, end-to-end representation learning that discovers task-relevant patterns from raw signals, and transparent interpretability through human-readable polynomial motifs rather than black-box architectures. **patX** searches over start/end indices, channels, and polynomial shape parameters to automatically learn discriminative patterns. Each discovered pattern induces a similarity feature based on root mean squared error to the corresponding signal segment. By combining guided search with a lightweight learner for pattern evaluation, **patX** achieves three key objectives simultaneously: it automates feature discovery like deep networks, learns compact representations optimized for the target task, and maintains full interpretability by producing inspectable patterns that clinicians can directly visualize and validate against domain knowledge.

We evaluate **patX** across heterogeneous biomedical problems: arrhythmia classification on MIT-BIH [19, 12], seizure classification on Bonn EEG [2], gene expression prediction from histone modification profiles (REMC) [7], and clinical prediction on MIMIC-IV [14], as well as a glucose forecasting task (AZT1D). We compare against **tsfresh** [6] and a convolutional neural network baseline under matched splits and metrics.

Our contributions are:

- We introduce **patX**, a dynamic polynomial pattern search that produces human-readable motifs and associated similarity features showcasing its strong performance on selected tasks against established baselines.
- We demonstrate the favorable interpretability of **patX** patterns by visualizing them and the areas at which they are active.
- We release **patX** as an official Python package on PyPI and actively maintain it (link: <https://pypi.org/project/patx/>).

2 Methods

2.1 Overview

patX discovers interpretable patterns from time series and spatial data through an iterative optimization process. Given input signals, **patX** searches for polynomial templates that, when com-

pared to signal segments via root mean squared error (RMSE), produce discriminative features for downstream prediction tasks. The method combines Bayesian optimization for efficient search with gradient boosting for pattern evaluation, ultimately producing a small set of human-interpretable patterns.

2.2 Pattern Representation

Consider a signal $\mathbf{x} \in \mathbb{R}^{T \times C}$ with T time points and C channels (for univariate signals, $C = 1$). A pattern p in patX is defined as a polynomial function over a contiguous interval:

$$p(t) = \sum_{i=0}^d c_i \cdot t^i, \quad t \in [-1, 1] \quad (1)$$

where d is the polynomial degree (default 3) and $\mathbf{c} = \{c_0, c_1, \dots, c_d\}$ are the coefficients. Each pattern is characterized by:

- Start position s and width w defining the interval $[s, s + w)$ in the signal
- Polynomial coefficients $\mathbf{c}$ defining the pattern shape
- Channel index k (for multivariate signals) specifying which channel to extract from

2.3 Feature Extraction

For each pattern p_j , we compute a similarity feature by measuring the RMSE between the pattern template and the corresponding signal segment:

$$f_j(\mathbf{x}) = \sqrt{\frac{1}{w_j} \sum_{i=0}^{w_j-1} \left(x_{s_j+i, k_j} - p_j \left(\frac{2i}{w_j-1} - 1 \right) \right)^2} \quad (2)$$

where x_{s_j+i, k_j} is the signal value at position $s_j + i$ in channel k_j . Before computing RMSE, we apply z-score normalization to both the signal segment and pattern template to ensure scale invariance.

2.4 Optimization Process

patX employs an iterative optimization strategy to discover patterns:

1. **Initialization:** Start with an empty pattern set $\mathcal{P} = \emptyset$ and optionally include domain-specific initial features (e.g., raw signal values)
2. **Pattern Proposal:** At iteration t , use Bayesian optimization (Optuna with Tree-Parzen Estimator) to propose B candidate patterns by sampling from the search space:

$$\theta = (s, w, \mathbf{c}, k) \in [0, T) \times [w_{min}, w_{max}] \times \mathbb{R}^{d+1} \times [0, C) \quad (3)$$

3. **Feature Matrix Construction:** For each candidate pattern, compute features $\mathbf{F} = [f_1(\mathbf{x}), \dots, f_j(\mathbf{x})]$ across all training samples
4. **Pattern Evaluation:** Train a LightGBM classifier/regressor using current features plus the candidate pattern feature. Evaluate on validation set to obtain performance metric $\mathcal{L}_{val}$

5. **Pattern Selection:** Accept the candidate with best validation performance if it improves upon the current best score:

$$\mathcal{P}_{t+1} = \begin{cases} \mathcal{P}_t \cup \{p^*\} & \text{if } \mathcal{L}_{val}(p^*) > \mathcal{L}_{best} \\ \mathcal{P}_t & \text{otherwise} \end{cases} \quad (4)$$

6. **Termination:** Stop when no improvement for $N_{patience}$ iterations or maximum iterations reached

2.5 Implementation Details

The core implementation uses the `PatternExtractor` class which:

- Converts inputs to float32 NumPy arrays with shape (N, C, T) for N samples
- Caches computed features to avoid redundant calculations
- Supports parallel pattern evaluation across multiple CPU threads
- Exports discovered patterns to JSON for reproducibility
- Integrates with scikit-learn API for seamless pipeline integration

Typical hyperparameters include 50–200 optimization rounds with 50–200 candidate patterns per round, targeting final pattern sets of ≤ 12 patterns to maintain interpretability.

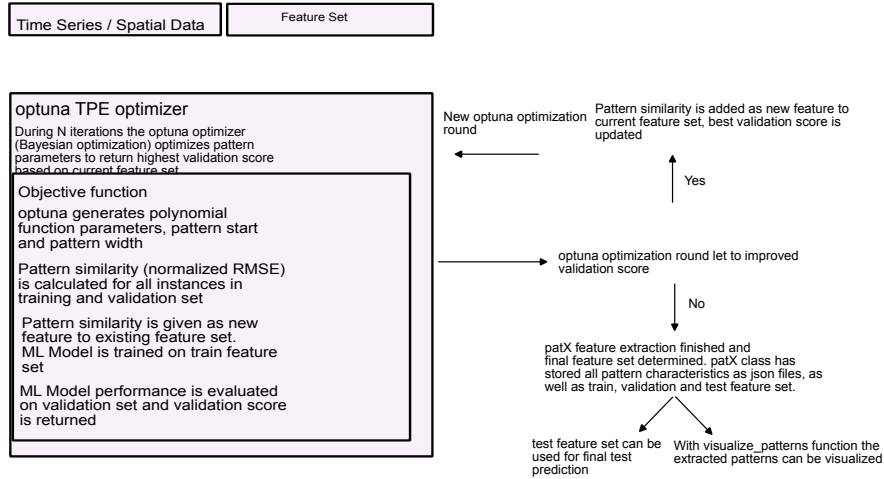


Figure 1: patX methodology overview: The iterative optimization process begins with time series or spatial data and an optional initial feature set. Optuna’s Tree-Parzen Estimator (TPE) optimizer proposes polynomial function parameters, pattern start positions, and widths. For each candidate pattern, similarity features (normalized RMSE) are computed across training and validation sets. These features augment the existing feature set, and a machine learning model evaluates performance on the validation set. If the validation score improves, the pattern is added to the feature set; otherwise, optimization continues. The process terminates when no improvement is observed, yielding a final feature set with stored pattern characteristics for test prediction and visualization.

patX is available as an official Python package on PyPI (<https://pypi.org/project/patx/>) and is actively maintained with comprehensive documentation and examples.

2.6 Data Pre-processing

AZT1D (glucose forecasting). We parse CGM records per subject and create supervised learning examples using lagged glucose features. Specifically, we construct 24 lag variables $\{g_{t-\ell}\}_{\ell=0}^{23}$ and define the regression target as the h -step change $g_{t+h} - g_t$ (where h is the prediction horizon). We drop rows with missing values induced by shifts. In addition, we pass two initial features to patX (raw glucose and time-of-day) so that patterns can augment a simple contextual baseline. No scaling is applied beyond NaN filtering. Link: <https://data.mendeley.com/datasets/gk9m674wcx/1>.

MITBIH (arrhythmia classification). We extract ECG beats from raw MIT-BIH recordings by centering 100-sample windows around annotated R-peak positions. Each beat corresponds to one of five arrhythmia classes: normal (N), left bundle branch block (L), right bundle branch block (R), atrial premature (A), or ventricular premature (V). We apply random undersampling to balance class distributions. This dataset originates from the PhysioNet MIT-BIH Arrhythmia Database (link: <https://www.kaggle.com/datasets/mondejar/mitbih-database>).

Bonn EEG (five-class EEG). We preprocess raw EEG recordings using wavelet decomposition with PyWavelets (pywt). Each signal undergoes 4-level wavelet decomposition using Daubechies-4 wavelets with periodization mode, yielding concatenated wavelet coefficients as features. The dataset contains five classes: seizure activity (S), focal area (F), healthy brain (N), eyes closed (O), and eyes open (Z). Link: <https://dataverse.csuc.cat/dataset.xhtml?persistentId=doi:10.34810/data490>.

REMC (ChIP-seq). We follow the same preprocessing approach as our prior work on PatternChrome [24]. For each cell line, we extract histone modification profiles from promoter regions and create binary gene expression targets (high/low expression). We form multi-series inputs by grouping histone marks (H3K4me1, H3K4me3, H3K27me3, etc.) as separate channels. patX searches over start/width/polynomial coefficients per histone mark to discover predictive modification patterns within promoter regions. Performance is measured via AUC. Data from Roadmap Epigenomics Consortium: https://egg2.wustl.edu/roadmap/web_portal/processed_data.html. Link of processed data: <https://gitlab.gwdg.de/MedBioinf/generegulation/patternchrome>

MIMIC-IV (EHR classification). We preprocess MIMIC-IV to derive one row per subject with 24 hourly samples per time series (e.g., heart rate, respiratory rate, blood pressure, labs) and a binary ARDS flag. Preprocessing includes: cleaning required columns, sorting by subject and hour, replacing infinities, capping extremes at the 0.1/99.9th percentiles, forward/backward filling within subjects, and padding/truncating to 24 hours per series. We rename series to clean identifiers (e.g., hourly heart rate 0..23). We then group columns by series name to create a multi-series input list, standardize each series on the train split, and provide age as an initial feature (standardized) to patX. Link: <https://physionet.org/content/mimiciv/3.1/>.

2.7 Datasets

- **AZT1D:** CGM and insulin pump data for type 1 diabetes forecasting (15/30 min horizons)
- **MITBIH:** ECG rhythm classification using the MIT-BIH Arrhythmia Database
- **Bonn EEG:** Five-class EEG dataset (Z,O,N,F,S) used in seizure research
- **REMC:** Histone modification profiles with gene expression labels from Roadmap Epigenomics
- **MIMIC-IV:** EHR time series for clinical prediction tasks

2.8 Baselines

We compare patX against two established approaches for time series analysis. The first baseline uses tsfresh for automated feature extraction combined with LightGBM, while the second employs a 1D convolutional neural network for end-to-end learning.

2.8.1 TSFRESH Baseline Implementation

For the tsfresh baseline, we use `MinimalFCParameters` to extract a compact descriptor set including moments, spectral features, and shape statistics. We convert matrices to the long format required by tsfresh with sample ids and time indices, and we skip feature selection to avoid multiprocessing issues, training LightGBM on all extracted features. Hyperparameters and evaluation are kept identical to patX and other baselines.

2.8.2 CNN Baseline Implementation

For the CNN baseline, we implement a 1D convolutional neural network using PyTorch with architecture tailored to each dataset’s input dimensions. The network consists of three convolutional layers with ReLU activations and max pooling, followed by dropout layers for regularization and fully connected layers for classification or regression. We use early stopping based on validation loss to prevent overfitting, with a patience of 10 epochs. The network is trained using Adam optimizer with learning rate 0.001 and batch size 32. For multi-class problems, we use cross-entropy loss; for regression, we use mean squared error loss. Hyperparameters and evaluation protocols are kept identical to patX and tsfresh baselines.

2.9 Evaluation

We split all datasets 75%/25% train/test with fixed seeds and preserve class balance. Classification uses area under the ROC curve (AUC) or accuracy; regression uses root mean squared error (RMSE). We also report the number of extracted patterns and wall-clock extraction time. Where applicable, we compute 95% CIs across seeds. Pattern search uses only training data. LightGBM is tuned via 5-fold CV on training features and retrained on the full train split. Results are averaged across 3 seeds. We fix the total pattern budget to ensure comparable runtime across datasets and perform ablations on degree, window length, and similarity. All baselines share the same data splits and evaluation protocol for a fair comparison.

3 Results

3.1 Performance Comparison

Table 1 presents the test set performance of patX + LightGBM, tsfresh + LightGBM, and CNN baselines across five biomedical datasets. We report mean and standard deviation.

On AZT1D (regression), CNN yields the lowest RMSE, while patX uses far fewer features and achieves lower inference time than CNN. On MITBIH and Bonn EEG, CNN leads in accuracy/AUC, but it requires orders of magnitude more features than patX (hundreds to thousands versus single digits), underscoring patX’s compactness.

On REMC and MIMIC-IV, patX achieves the best performance while using a small number of patterns. In REMC, patX outperforms both tsfresh and CNN with roughly single-digit patterns

Table 1: Performance comparison across biomedical datasets showing mean $\pm$ standard deviation across folds/subjects.

Dataset	Method	Performance	# Features	Time (s)
AZT1D	PATX	18.868 ± 0.592	3.7 ± 1.2	27.4 ± 6.2
	TSFRESH	20.010 ± 0.812	10.0 ± 0.0	1.9 ± 0.2
	CNN	16.585 ± 0.304	24.0 ± 0.0	51.2 ± 5.4
MITBIH	PATX	0.893 ± 0.005	8.6 ± 1.5	113.3 ± 19.4
	TSFRESH	0.823 ± 0.004	10.0 ± 0.0	65.4 ± 142.0
	CNN	0.958 ± 0.003	100.0 ± 0.0	243.8 ± 303.1
Bonn EEG	PATX	0.700 ± 0.073	4.6 ± 1.1	44.6 ± 10.0
	TSFRESH	0.584 ± 0.036	10.0 ± 0.0	0.7 ± 0.1
	CNN	0.788 ± 0.040	4101.0 ± 0.0	38.5 ± 10.7
REMC	PATX	0.928 ± 0.024	9.0 ± 2.2	92.4 ± 192.8
	TSFRESH	0.874 ± 0.030	10.0 ± 0.0	5.8 ± 0.1
	CNN	0.889 ± 0.029	1000.0 ± 0.0	381.5 ± 1308.7
MIMIC-IV	PATX	0.778 ± 0.002	5.2 ± 1.3	30.0 ± 7.2
	TSFRESH	0.769 ± 0.004	10.0 ± 0.0	1.2 ± 0.0
	CNN	0.773 ± 0.009	697.0 ± 0.0	36.6 ± 4.4

and competitive runtime. In MIMIC-IV, patX attains the highest accuracy/AUC with a handful of features, whereas CNN uses considerably more learned filters.

Overall, the results show a consistent trade-off: CNN can be strong on some tasks when large learned representations are allowed, but patX reaches competitive or superior performance with drastically fewer features and favorable wall-clock times, while providing human-readable patterns for interpretation.

3.2 Pattern Discovery Characteristics

We summarize global properties of the discovered motifs across datasets: the number of patterns per run, their start indices and widths, and aggregate counts. For each dataset, we aggregate all extracted patterns from the exported JSON artifacts and report summary statistics in Table 2. The table lists, per dataset, the number of runs processed, total pattern count, per-run mean $\pm$ std of pattern counts, the median start index with range, and the median $\pm$ std of window width.

Table 2: Summary statistics of extracted patterns across datasets. Patterns/run is the per-run mean $\pm$ std; start shows median [min, max]; width shows median $\pm$ std.

Dataset	Runs	Total patterns	Patterns/run	Start (med [min,max])	Width (med $\pm$ std)
AZT1D	24	96	4.0 ± 1.5	4 [0, 22]	6 ± 4.9
BONN	1	4	4.0 ± 0.0	382 [274, 755]	2370 ± 866.3
MIMIC	1	5	5.0 ± 0.0	11 [0, 15]	8 ± 6.7
MITBIH	1	8	8.0 ± 0.0	56 [4, 92]	5 ± 5.2
REMC	56	485	8.7 ± 2.1	30 [0, 198]	73 ± 54.7

3.3 Interpretability Case Studies

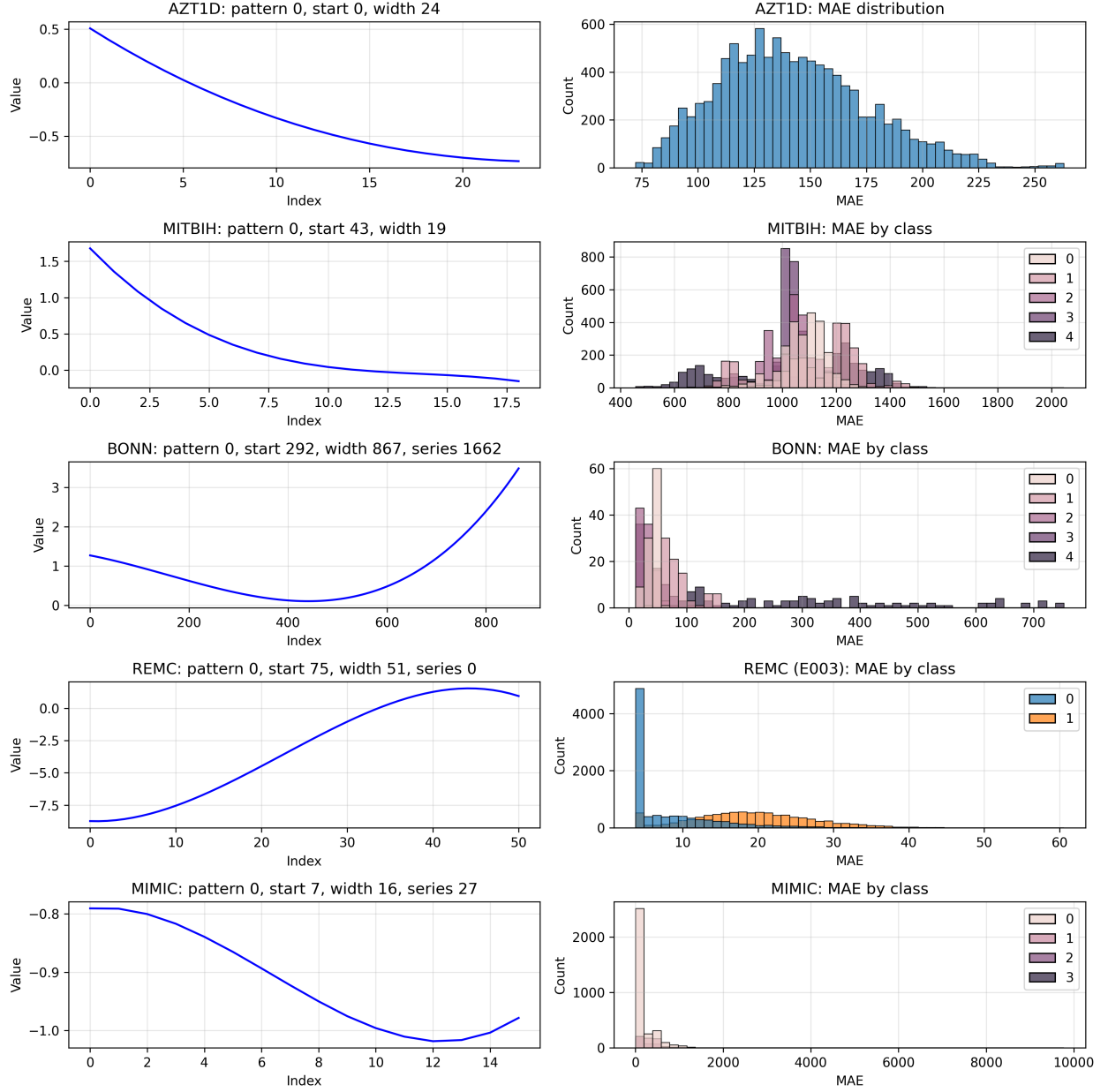


Figure 2: First discovered pattern per dataset (left: template shape; right: value distribution). For REMC and AZT1D, we use the first pattern of the first sample.

To demonstrate patX’s interpretability advantage, we examine how discovered patterns behave across different signal types. Figure 3 shows the first pattern discovered on MITBIH overlaid in a *single* plot on a normal ECG and an arrhythmic ECG (both z-scored for comparability). The pattern spans the highlighted indices and captures a characteristic morphology that separates beat types. On the normal beat, the template aligns with a smooth recovery segment; on the arrhythmic beat, the same region exhibits irregular dynamics, evidencing the pattern’s discriminative behavior.

This visualization illustrates a key benefit of patX: unlike black-box methods, the discovered

patterns can be directly inspected and related to domain knowledge. Clinicians can examine where patterns activate, understand their morphological characteristics, and validate their clinical relevance. The polynomial representation provides smooth, interpretable templates that capture essential signal features while remaining human-readable.

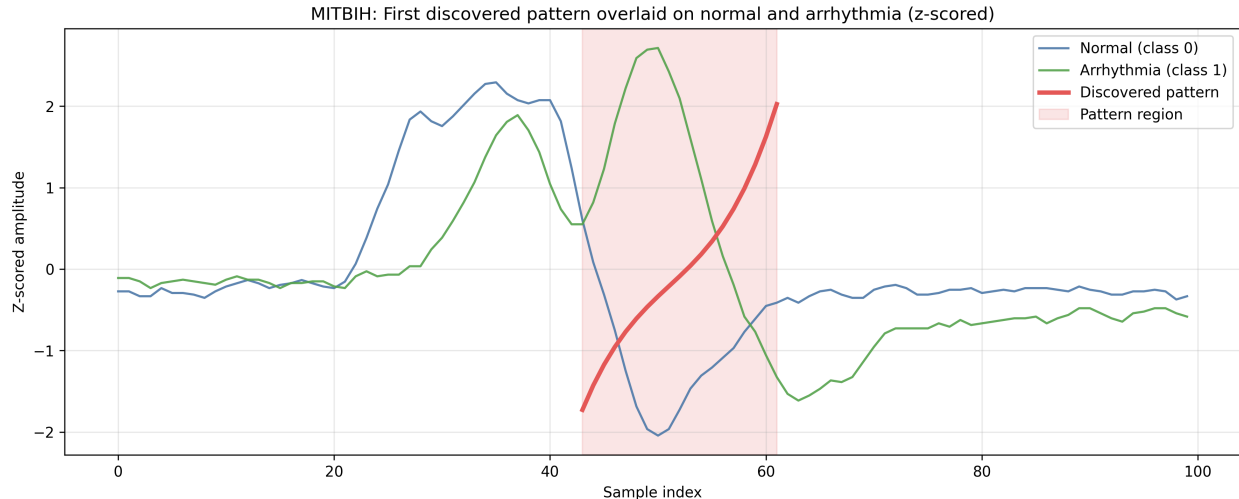


Figure 3: First discovered MITBIH pattern overlaid on a normal ECG and an arrhythmic ECG in a single plot (signals and template z-scored). The red line is the pattern template; the shaded region marks the active indices. The pattern exhibits distinct alignment across beat types, supporting interpretability.

3.4 Decision Boundary Visualization

We study a two-pattern decision boundary for REMC E003: we compute the RMSE feature for the first two extracted patterns for each sample and fit a small gradient boosting classifier (LightGBM) on this 2D feature space. Using 18,421 samples with a 75/25 train/test split, we achieve 76.4% training accuracy and 74.6% test accuracy on the balanced binary classification task (2,304 vs 2,302 samples per class).

Figure 4 visualizes the decision boundary in the two-dimensional feature space of RMSE values. Pattern 1 (start=44, width=55) yields RMSE values ranging from 0.797 to 2.761, while Pattern 2 (start=12, width=94) produces a tighter range from 0.383 to 2.074. The background regions show the predicted class probabilities, with semi-transparent data points (X markers) overlaid to reveal the actual class labels. This visualization demonstrates how just two interpretable pattern features can effectively separate gene expression classes while remaining directly inspectable and clinically meaningful.

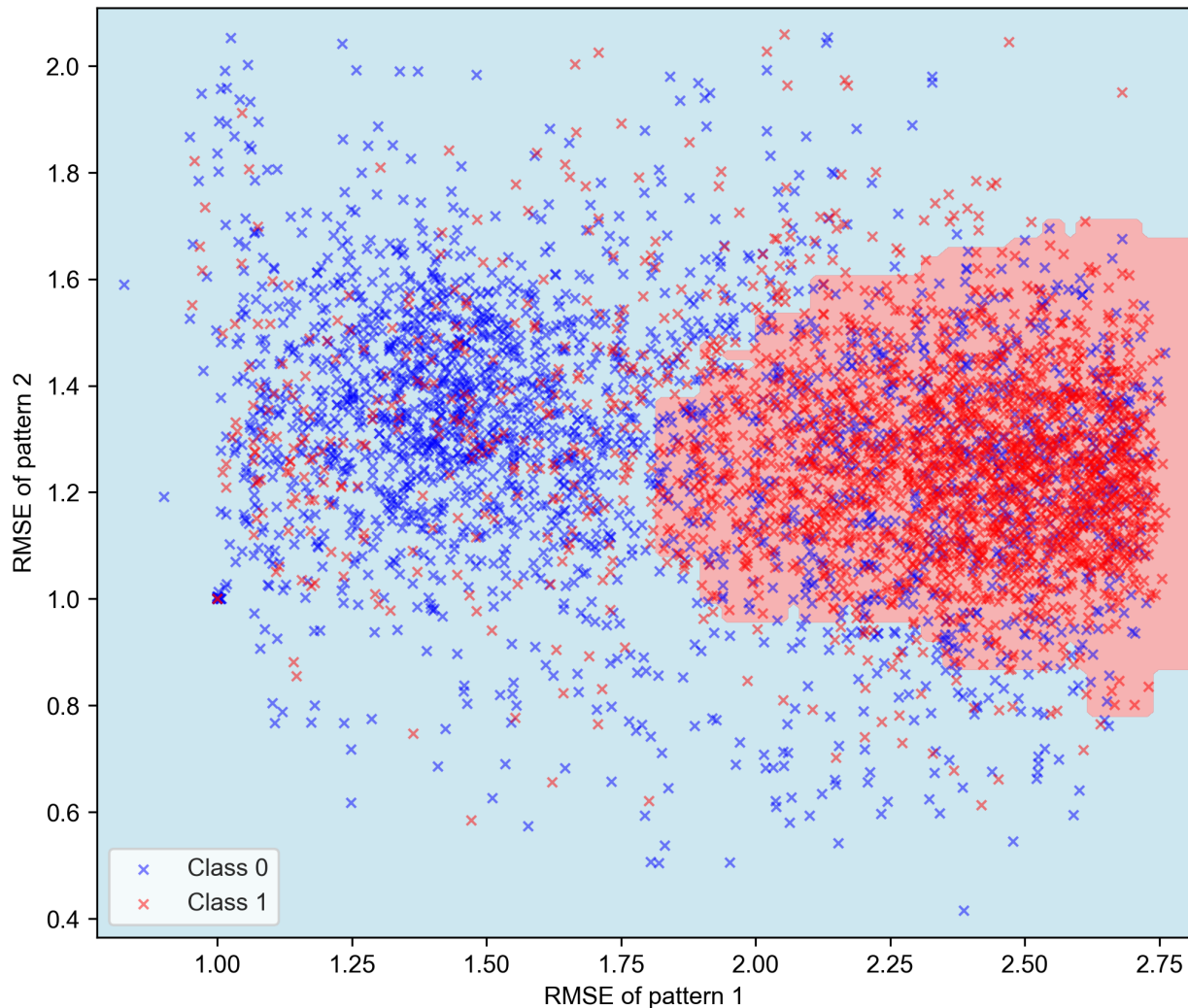


Figure 4: REMC E003: Decision boundary visualization in 2D feature space using RMSE values from the first two extracted patterns. Background colors indicate predicted class regions with 60% opacity, while X markers show actual test samples colored by true class labels. The classifier achieves 74.6% test accuracy using only these two interpretable pattern features.

3.5 Ablation Study

4 Discussion

5 Conclusion

We introduced patX, an interpretable pattern extraction framework bridging the gap between hand-crafted descriptors and opaque deep models. Through extensive experiments, we demonstrated that a small set of automatically discovered motifs can match or exceed the predictive power of deep learning across heterogeneous biomedical domains, while offering orders-of-magnitude reductions in feature count and inference cost. Future extensions will explore adaptive pattern lengths, semi-supervised pretraining on large unlabeled corpora, and tighter integration with interactive

visualization tools to facilitate clinical adoption.

References

- [1] Thivya Anbalagan, Malaya Kumar Nath, Dhurairajan Vijayalakshmi, and Archana Anbalagan. Analysis of various techniques for ecg signal in healthcare, past, present, and future. *Biomedical Engineering Advances*, 6:100089, 2023.
- [2] Ralph G Andrzejak, Klaus Lehnertz, Florian Mormann, Christoph Rieke, Peter David, and Christian E Elger. Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: dependence on recording region and brain state. *Physical Review E*, 64(6):061907, 2001.
- [3] Yaqoob Ansari, Omar Mourad, Khalid Qaraqe, and Erchin Serpedin. Deep learning for ecg arrhythmia detection and classification: an overview of progress for period 2017–2023. *Frontiers in Physiology*, 14:1246746, 2023.
- [4] Marília Barandas, Duarte Folgado, Letícia Fernandes, Sara Santos, Mariana Abreu, Patrícia Bota, Hui Liu, Tanja Schultz, and Hugo Gamboa. Tsfel: Time series feature extraction library. *SoftwareX*, 11:100456, 2020.
- [5] Ahmad Chaddad, Yihang Wu, Reem Kateb, and Ahmed Bouridane. Electroencephalography signal processing: A comprehensive review and analysis of methods and techniques. *Sensors*, 23(14):6434, 2023.
- [6] Maximilian Christ, Nils Braun, Julius Neuffer, and Alexander W Kempa-Liehr. Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package). *Neurocomputing*, 307:72–77, 2018.
- [7] Roadmap Epigenomics Consortium et al. Integrative analysis of 111 reference human epigenomes. *Nature*, 518(7539):317–330, 2015.
- [8] Mohammad Ennab and Hamid Mcheick. Enhancing interpretability and accuracy of ai models in healthcare: a comprehensive review on challenges and future directions. *Frontiers in Robotics and AI*, 11:1444763, 2024.
- [9] Ben D Fulcher. Feature-based time-series analysis. In *Feature engineering for machine learning and data analytics*, pages 87–116. CRC press, 2018.
- [10] Ben D Fulcher and Nick S Jones. hctsa: A computational framework for automated time-series phenotyping using massive feature extraction. *Cell systems*, 5(5):527–531, 2017.
- [11] Mehmet Ali Gelen, Prabal Datta Barua, Irem Tasci, Gulay Tasci, Emrah Aydemir, Sengul Dogan, Turker Tuncer, and UR Acharya. Novel accurate classification system developed using order transition pattern feature engineering technique with physiological signals. *Scientific Reports*, 15(1):15278, 2025.
- [12] Ary L Goldberger, Luis AN Amaral, Leon Glass, Jeffrey M Hausdorff, Plamen Ch Ivanov, Roger G Mark, Joseph E Mietus, George B Moody, Chung-Kang Peng, and H Eugene Stanley. Physiobank, physiotoolkit, and physionet: components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000.

- [13] Trent Henderson and Ben D Fulcher. An empirical evaluation of time-series feature sets. In *2021 International Conference on Data Mining Workshops (ICDMW)*, pages 1032–1038. IEEE, 2021.
- [14] Alistair EW Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom J Pollard, Sicheng Hao, Benjamin Moody, Brian Gow, et al. MIMIC-IV, a freely accessible electronic health record dataset. *Scientific data*, 10(1):1, 2023.
- [15] Jeong Min Lee and Milos Hauskrecht. Modeling multivariate clinical event time-series with recurrent temporal mechanisms. *Artificial intelligence in medicine*, 112:102021, 2021.
- [16] Pengfei Liu, Xiaoming Sun, Yang Han, Zhishuai He, Weifeng Zhang, and Chenxu Wu. Arrhythmia classification of LSTM autoencoder based on time series anomaly detection. *Biomedical Signal Processing and Control*, 71:103228, 2022.
- [17] Carl H Lubba, Sarab S Sethi, Philip Knaute, Simon R Schultz, Ben D Fulcher, and Nick S Jones. catch22: Canonical time-series characteristics: Selected through highly comparative time-series analysis. *Data mining and knowledge discovery*, 33(6):1821–1852, 2019.
- [18] Riccardo Miotto, Fei Wang, Shuang Wang, Xiaoqian Jiang, and Joel T Dudley. Deep learning for healthcare: review, opportunities and challenges. *Briefings in bioinformatics*, 19(6):1236–1246, 2018.
- [19] George B Moody and Roger G Mark. The impact of the MIT-BIH arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3):45–50, 2001.
- [20] Mohammad Amin Morid, Olivia R Liu Sheng, and Joseph Dunbar. Time series prediction using deep learning methods in healthcare. *ACM Transactions on Management Information Systems*, 14(1):1–29, 2023.
- [21] Mohammad Amin Morid, Olivia R Liu Sheng, Kensaku Kawamoto, and Samir Abdelrahman. Learning hidden patterns from patient multivariate time series data using convolutional neural networks: A case study of healthcare cost prediction. *Journal of Biomedical Informatics*, 111:103565, 2020.
- [22] Nina Moutonnet, Gregory Scott, and Danilo P Mandic. Augmentation of EEG and ECG time series for deep learning applications: Integrating changepoint detection into the IAAFT surrogates. *arXiv preprint arXiv:2504.03761*, 2025.
- [23] Abhidnya Patharkar, Fulin Cai, Firas Al-Hindawi, and Teresa Wu. Predictive modeling of biomedical temporal data in healthcare applications: review and future directions. *Frontiers in Physiology*, 15:1386760, 2024.
- [24] Niels Benjamin Paul, Jonas Chanrithy Wolber, Malte Lennart Sahrhage, Tim Reißbarth, and Martin Haubrock. Prediction of gene expression using histone modification patterns extracted by particle swarm optimization. *Bioinformatics*, 41(2):btaf033, 2025.
- [25] Gašper Petelin, Gjorgjina Cenikj, and Tome Eftimov. Towards understanding the importance of time-series features in automated algorithm performance prediction. *Expert systems with applications*, 213:119023, 2023.

- [26] Kenneth D Roe, Vibhu Jawa, Xiaohan Zhang, Christopher G Chute, Jeremy A Epstein, Jordan Matelsky, Ilya Shpitser, and Casey Overby Taylor. Feature engineering with clinical expert knowledge: A case study assessment of machine learning model complexity and performance. *PloS one*, 15(4):e0231300, 2020.
- [27] Jennifer E Rood, Aidan Maartens, Anna Hupalowska, Sarah A Teichmann, and Aviv Regev. Impact of the human cell atlas on medicine. *Nature medicine*, 28(12):2486–2496, 2022.
- [28] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.
- [29] Will Ke Wang, Ina Chen, Leeor Hershkovich, Jiamu Yang, Ayush Shetty, Geetika Singh, Yihang Jiang, Aditya Kotla, Jason Zisheng Shang, Rushil Yerrabelli, et al. A systematic review of time series classification techniques used in biomedical applications. *Sensors*, 22(20):8016, 2022.
- [30] Amit Kumar Yadav, Sanjay Kumar Banerjee, Bhabatosh Das, and Kumardeep Chaudhary. systems biology and omics approaches for understanding complex disease biology. *Frontiers in Genetics*, 13:896818, 2022.